Expert Systems With Applications 270 (2025) 126354

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Review

Federated learning framework based on trimmed mean aggregation rules

Tianxiang Wang, Zhonglong Zheng *, Feilong Lin

College of Mathematics and Computer Science, Zhejiang Normal University, China

ARTICLE INFO

Keywords:
Federated learning
Mean
Tmean
Byzantine resilient

ABSTRACT

This paper investigates the issue of information security against the context of a distributed learning framework. In particular, it is believed that clients will often be attacked by Byzantine nodes and poisoning in federated learning, aggregation rules are used for protecting the model from attacking. However, classical aggregation methods such as Krum and Mean are not efficient enough to deal with Byzantine attacks, in which general deviations and multiple clients are attacked at the same time. A new aggregation rule, Tmean is hereby introduced to the federated learning algorithm. The federated learning framework is proposed based on Byzantine resilient aggregation algorithm. The Tmean rule is derived from Mean by appropriately trimming some of the values before averaging them. Theoretical analysis and understanding of Tmean are conducted, and the efficacy of which is confirmed by extensive experiments.

Contents

1. Introduction 2
2. Related work 2
3. Preliminary and problem setup 3
   3.1. Federated learning setting 3
   3.2. Byzantine resilience 4
   3.3. Clipping mean in FL 4
4. FL framework with robust aggregation 5
   4.1. Krum 5
   4.2. Mean-based aggregation rules 5
   4.3. Tmean time complexity analysis 5
5. Convergence analysis 6
6. Experiments 6
   6.1. Experiment setting 6
   6.2. Gaussian attack 6
   6.3. Omniscient attack 7
   6.4. General attack with multiple servers 7
   6.5. Experiment conclusion 8
7. Conclusions 8
   Declaration of competing interest 9
   Acknowledgments 9
   Appendix. 9
   Appendix. Data availability 10
   References 10

* Corresponding author.
E-mail addresses: wangtianxiang@zjnu.edu.cn (T. Wang), zhonglong@zjnu.edu.cn (Z. Zheng), bruce_lin@zjnu.cn (F. Lin).

https://doi.org/10.1016/j.eswa.2024.126354
Received 2 August 2022; Received in revised form 17 December 2024; Accepted 29 December 2024
Available online 10 January 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

Federated learning (FL), a subset of distributed machine learning, it has received more and more attention recently. FL has emerged as one promising approach enabling clients to learn a shared model collaboratively using decentralized and private data on each client node. To this end, it is of central importance to keep the node information secured in FL.

Unfortunately, FL is extremely vulnerable to software/hardware errors and adversarial attacks. In particular, the Byzantine attack from distributed systems has emerged as a node attack sample for the federated learning model.

To ensure the FL model resistance to Byzantine attacks, research focuses are placed on *how to introduce aggregation rules into the gradient information iteration process*, and*how to ensure that this aggregation rule can make the data robust*. Classical approaches to avoid the Byzantine failures would employ the state machine replication strategy (Alistarh, Allen-Zhu, & Li, 2018). Which can be roughly categorized into two ways in distributed machine learning: (1) the processes agree on a sample of data based on which the clients update their local parameter vectors, which demands transmitting data samples to each individual node, resulting in high costs. (2) the clients agree on how the parameter vector should be updated, which is not reliable neither, as it cannot be effectively detected whether clients are trustworthy or not from the mixed Byzantine vectors (Blanchard, Mhamdi, Guerraoui, & Stainer, 2017). The attackers can know any information about the FL process, and can use any vectors to initiate the attack during the node information transmission (Yin, Chen, Kannan, & Bartlett, 2018). More specifically, the data between machines can be replaced by any value.

In the case of attacks from Byzantine nodes, the FL models minimize the influence of Byzantine attack on the data model relying on robust aggregation rules (Bottou, 2010). For instance, $\mathrm{Krum}(\cdot)$ is a strong aggregation rule designed to identify an honest node and effectively prevent Byzantine attacks in most cases (Blanchard et al., 2017). However, if the Byzantine node attacks are changed from original single miner node attack to multiple servers attacks, $\mathrm{Krum}(\cdot)$ fails to ensure the learning process robustness to noisy data. Simple $\mathrm{Mean}(\cdot)$ aggregation rule, as another class of aggregation rules, can maintain learning robustness if not attacked by Byzantine nodes. However, Byzantine attacks will largely deviate the update direction of node gradient information from the original function using simple mean-based aggregation rules. Some variants of mean-based aggregation rules. such as *geometric median* (Blanchard et al., 2017) and *marginal median* (Alistarh et al., 2018), are classical methods to solve the problem of the Byzantine node attack. But they are not much more robust than simple mean-based aggregation rules in the case of some large deviation Byzantine attacks. The main difficulty preventing mean-based aggregation rules from robust is the unstable data distribution (Jin, Huang, He, Wu, & Dai, 2020). Which can be tackled if the data are averaged by trimming a part of the data and then imported into the aggregation rules.

Most FL approaches are built upon the Stochastic Gradient Descent (SGD) algorithm (Castro, Liskov, et al., 1999) or its variants, and the statistical properties of SGD such as convergence are well developed. SGD is also employed by the hereby proposed approach; and the typical iterative process of SGD algorithm in the distributed system is represented by Figure (a). First, a client, known as a miner node, estimates the gradient of the node; makes an estimate of the deviation between the estimated information and the ideal gradient information (El-Mhamdi, Guerraoui, Guirguis, Hoang, & Rouault, 2020); then transmits the information to the server node in the network; and finally updates the gradient there. To this end, the miner node network transmits the information to the master server, and the master server then passes the gradient update information through a series of aggregation operations. The server transmits the information to the distribution network when the aggregation conditions are met, which is then transmitted to the miner nodes. This is a cyclic process, with the gradient information continuously updated in the entire network (Li, Andersen, Smola, & Yu, 2014).

In this paper, a new aggregation rule, $\mathrm{Tmean}(\cdot)$ is mainly proposed by the trimming part of the data before average operation. Theoretical understandings of the aggregation rule is provided, especially the reason for their robustness to the Byzantine attack. It is concluded based on attack experiments and mathematical proofs that appropriately trimming the original data and averaging makes the model more robust from the decentralized data.

Specifically, Section 3introduces the federated learning Byzantine distributed system model. Briefly describes the working principle of SGD, and summarizes the update iteration rules based on aggregation rules. The concept of Byzantine resilience, and the conditions to satisfy the aggregation rules of Byzantine resilience are then presented. Section 4 clarifies the concept of trimmed mean, also the rigorous theoretical proof and understanding of $\mathrm{Tmean}(\cdot)$-based aggregation rules. Then, in Section 5, the convergence of the proposed federated learning aggregation rules is proven. And in Section 6, experiments are conducted by Gaussian attack, omniscient attack and multiple servers attack. Under these attacks, $\mathrm{Tmean}(\cdot)$-based FL aggregation rules can still maintain their robustness. These experiments thus validate the effectiveness of the hereby proposed approaches. It is important to note that we are only considering Byzantine attacks on miner nodes here, although we can later extend this to server nodes and even global node defense.

Contributions:. (1) We present a new aggregation rule, $\mathrm{Tmean}(\cdot)$, to the Byzantine resilient federated learning algorithm, and propose federated learning frameworks based on aggregation algorithm. Our proposed approaches are shown to be robust to Byzantine attacks.

(2) We provide rigorous theoretical proof and understanding of our approaches and aggregation rules. To the best of our knowledge, these theorems and theoretical understandings are for the first time contributed to the community. Critically, we make convergence certificates and prove that $\mathrm{Tmean}(\cdot)$ can converge in the general convex optimization setting.

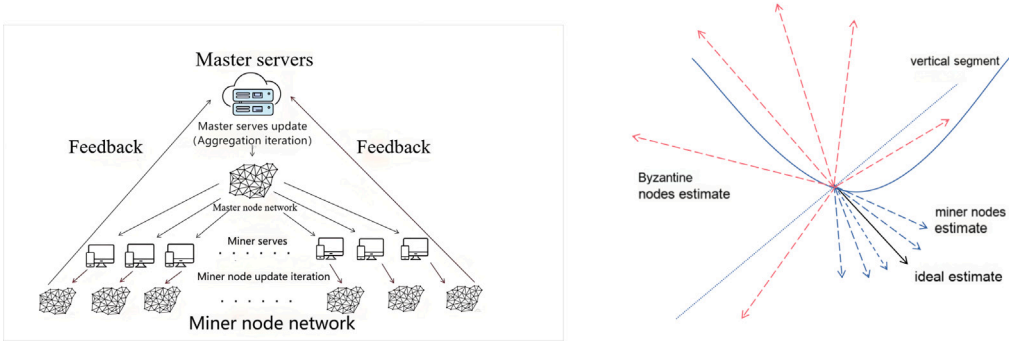
(3) Empirically, we demonstrate that the effectiveness of our approaches can make the FL model robust to Byzantine attack. In addition, our experiments also show that in the case of simultaneous attack of multiple servers, both Cifar 10, Cifar 100 and MNIST data sets have higher accuracy.

We firstly describes the aggregation rules based on federated learning, and explains the actual operation process of the aggregation rules in detail, then the aggregation rules are discussed, and their feasibility is analyzed through the optimization algorithm, and finally in the experimental part, for node attacks in different situations. The paper points out that the aggregation method based on Tmean has stronger practicability, and the existing aggregation methods are compared and summarized.

This paper is divided into the following main parts: Firstly, the proposed anti-Byzantine model and the main existing problems; secondly, we discuss the existing anti-Byzantine aggregation methods and the related methods proposed by me; secondly, we spend a chapter on the complexity and convergence analysis of the $\mathrm{Tmean}(\cdot)$ method; finally, the paper focuses on the experimental effect proof.

2. Related work

Federated learning. Federated learning has become a prominent distributed learning paradigm. In Jin et al. (2020), the authors proposed Stochastic-Sign SGD, a parameter estimation method with convergence guarantees. It uses a gradient compressor based on random symbols to unify that the gradient is updated in the framework. The FedAvg algorithm was first proposed by Konečný et al. (2016), and many scholars subsequently improved it based on this algorithm. Karimireddy et al. (2020) pointed out that FedAvg can be improved by adding one additional parameters control variables to correct the client drift, and proved that FedAvg may be seriously affected by the gradient difference of different clients, and may even be slower than SGD.



(a) Federated learning gradient update iteration (b) SGD gradient update

Fig. 1. Fig. 1(a) is the procession of FL gradient updating. It consists of four parts: master server, master server distributed network, miner servers and miner network. Fig. 1(b) represents the schematic diagram of the gradient update direction: blue dashed lines represent the estimated gradients of miner nodes; red dashed lines represent Byzantine update gradients under the attack of Byzantine nodes; black solid line represents the ideal gradient.

Byzantine attack in FL. To make the secure transmission of the master server node, the FL models have to deal with the Byzantine attack. Yin et al. (2018) showed that certain aggregation rules in the master server node can ensure the robustness of the data. Blanchard et al. (2017) pointed out that $\mathrm{Krum}(\cdot)$ can be used to make the robustness of data by computing the local sum of squared Euclidean distance to the other candidates, and outputting the one with minimal sum. When the gradient is updated to the saddle point, there is no guarantee that the SGD algorithm converges to the global optimum. Some scholars have proposed ByzantinePGD (Yin, Chen, Kannan, & Bartlett, 2019), which can escape saddle points and false local minimums, and can converge to an approximate true local minimum with low iteration complexity. Later, in the strong anti-Byzantine model, some scholars carried out poisoning attacks on Byzantine robust data, from the original data collection process to the subsequent information exchange process, and at the same time, these attacks were defended accordingly, in Zhao, Sun, Fang, Wang, and Jiang (2021). The research on the Byzantine structure model of federated learning has been expanded once again (Zhao et al., 2021).

Attack and defence. In the framework of federated learning, miner nodes are usually attacked, such as Byzantine node attacks, poisoning attacks (Zhao et al., 2021), gradient leakage (Wei et al., 2020), etc. There exist various defense methods also, such as robust aggregation, secure aggregation, encryption, etc. Our paper mainly studies Byzantine node attacks. Byzantine node attacks can usually be summarized into two types: (1) To update gradient information of nodes, nodes are replaced by Byzantine nodes, and normal worker nodes cannot make judgments so that the gradient estimates deviate from the actual gradient update direction. (2) During the gradient process, the local node suffering from the interference of the Byzantine node, cannot reach a consensus with the master node, making the entire process unable to proceed normally (Lamport, Shostak, & Pease, 2019). $\mathrm{Krum}(\cdot)$ is a popular aggregation rule to deal with these node attacks. We shall propose $\mathrm{Tmean}(\cdot)$ as alternative aggregation rules.

Federated learning against backdoor attacks. Federated learning (FL) is a promising approach for decentralized machine learning, enabling multiple participants to collaboratively train models while preserving data privacy. However, the architecture is susceptible to backdoor attacks, where malicious participants inject harmful updates, compromising model integrity and security. This vulnerability necessitates robust defenses to ensure the reliability of FL systems. Recent studies have focused on various strategies to enhance FL security against such threats. For instance, techniques like robust aggregation methods, including Krum and Trimmed Mean, have been developed to filter out malicious contributions. Additionally, the incorporation of differential privacy helps obscure individual updates, making it difficult for attackers to execute successful backdoor strategies. Research also explores adversarial training to prepare models for potential attacks, and the application of ensemble methods to strengthen resilience against poisoned updates. Furthermore, blockchain technology is being investigated for its potential to ensure transparency and accountability in the update process. Notable articles in this field include "Robust Federated Learning against Backdoor Attacks (Ranjan, Gupta, Coro, & Das, 2023)", "Differential Privacy in Federated Learning: A Survey (El Ouadrhiri & Abdelhadi, 2022)", and "Blockchain for Federated Learning: Security and Privacy Perspectives (Zhu, Wang, & Liu, 2021)", which collectively highlight the importance of multi-faceted approaches in safeguarding federated learning systems against backdoor attacks.

In this article, we will only consider the measures for effective defense using the Byzantine aggregation method when a Byzantine node attacks a client node. Of course, according to the recent progress of work, we can consider the defense method when the server master node is attacked, of course, this can be used in more models, such as decentralized blockchain modules.

3. Preliminary and problem setup

3.1. Federated learning setting

Federated learning was first proposed in Konečný et al. (2016), where the prevalent asynchronous SGD is used to update a global model in a distributed fashion. A pioneering work in this field proposed the currently most widely used algorithm FedAvg (Lugosi & Mendelson, 2021), which is also the first synchronous algorithm dedicated to federated setting. Recent studies attempt to expand federated learning with the aim of providing learning in more diverse and practical environments such as multi-task learning, generative models, continual learning, and data with noisy labels (Hu & Sung, 2004). However, these algorithms may obtain suboptimal performance when miners participating in FL have non-independently and identical distributions (non-i.i.d.).

While the convergence of FedAvg on such settings was initially shown by experiments in Helmers, Jing, Qin, and Zhou (2024), it does not guarantee performance as good as that in an i.i.d. setting. These algorithms pointed out the issue have major limitations, such as privacy violation by partial global sharing of local data (Lugosi & Mendelson, 2021) or no indication of improvement over baseline algorithms such as FedAvg. Our paper focuses on a general federated setting. We can further refine this iterative process through Fig. 2.

In order to better study the problem of aggregation in federated learning, we consider the following optimization problem:

$$\min_{x\in\mathbb{R}^d} F(x), \tag{1}$$

where $F(x) = \mathsf{E}_{\xi\sim D}[f(x;\xi)]$ is a smooth convex function, ξ is sampled from some unknown distribution D. Ideally, the problem (1) can be

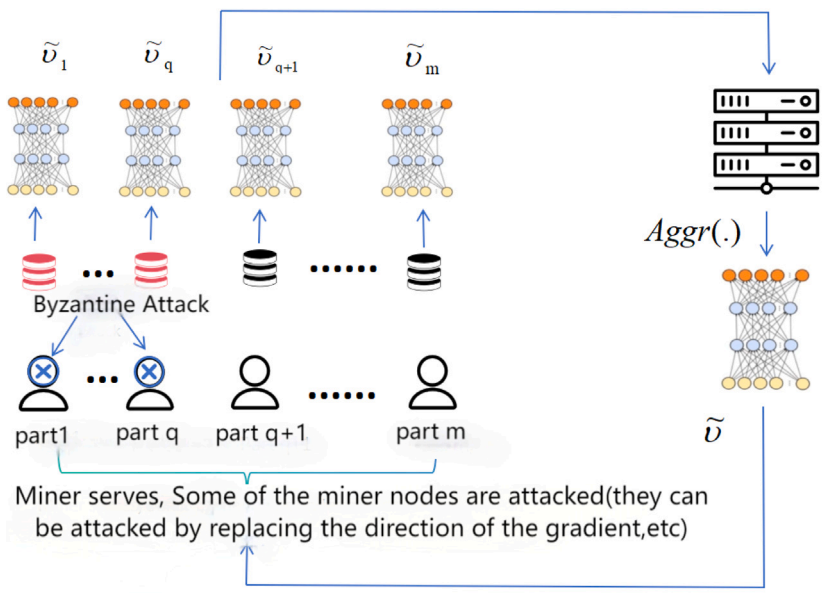


Fig. 2. Fig. 2 is a threat model of federated learning node information aggregation. On the left is the gradient iteration information of miner nodes, where the number of attacked nodes is q and the number of miner nodes is m. The miner nodes transmit the iterative information to the central server through gradient update, where the information is aggregated and finally returned to the miner nodes. This is a cyclic process, which constantly updates the node information to make the system stable.

solved by the gradient descent method as

$$x^{(t+1)} \leftarrow x^{(t)} - \gamma^{(t)} \nabla F(x^{(t)}), \tag{2}$$

where $\gamma^{(t)}$ is the learning rate at tth round. We assume that there exists at least one minimizer of $F(x)$, denoted by x^*, that satisfies $\nabla F(x^*) = 0$.

The problem (1) is solved in a distributed manner with m miner nodes, and up to q of them may be Byzantine nodes. The detailed algorithm of distributed synchronous SGD with aggregation rule $\mathrm{Aggr}(\cdot)$ is shown in Algorithm 1. In each iteration, each miner samples n i.i.d. data points from the distribution $\mathcal{D}$, and computes the gradient of the local empirical loss. Using a certain aggregation rule $\mathrm{Aggr}(\cdot)$, the server collects and aggregates the gradients sent by the miners, and estimate the gradient through $\nabla F(x^{(t)}) \approx \mathrm{Aggr}(v_1, v_2, \ldots, v_m)$. Without Byzantine failures, the kth worker calculates $v_k^{(t)} \sim G^{(t)}$, where $G^{(t)} = \nabla F(x^{(t)}, \xi)$. When there exist Byzantine faults, $v_k^{(t)}$ can be replaced by any arbitrary value $\tilde{v}_k^{(t)}$, and hence

$$\nabla F(x^{(t)}) \approx \mathrm{Aggr}(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m). \tag{3}$$

Server:
Initialize $x_0 \leftarrow$ `rand()`;
for $t = 0, 1, \ldots, T$ **do**
  Broadcast $x^{(t)}$ to all the workers;
  Wait until all the gradients $\tilde{v}_1^{(t)}, \tilde{v}_2^{(t)}, \ldots, \tilde{v}_m^{(t)}$ arrive;
  Compute $G^{(t)} = \mathrm{Aggr}(\tilde{v}_1^{(t)}, \tilde{v}_2^{(t)}, \ldots, \tilde{v}_m^{(t)})$;
  Update the parameter $x^{(t+1)} \leftarrow x^{(t)} - \gamma^{(t)} G^{(t)}$;
end

Worker:
for $t = 0, 1, \ldots, T$ **do**
  Receive $x^{(t)}$ from the server;
  Compute and send the local randomized gradient $v_k^{(t)} = \nabla F(x^{(t)}, \xi_k)$ to the server;
end

Algorithm 1: Distributed synchronous SGD with aggregation server.

3.2. Byzantine resilience

In the following we introduce the concept of *Byzantine resilience*. Suppose that in a specific iteration, the correct vectors $v_1, v_2, \ldots, v_m$ are i.i.d. samples drawn from the random variable $G = \nabla f(x; \xi_k)$, where $\mathsf{E}[G] = g$ is an unbiased estimator of the gradient based on the current parameter x. Thus, $\mathsf{E}[v_k] = \mathsf{E}[G] = g$ for $k \in \{1, 2, \ldots, m\}$. We simplify the notations by omitting the index of iteration t. The following Definition 1 defines the concept of Δ-Byzantine resilience. For simplicity, we sometimes use Byzantine resilient in short when the concrete value of Δ is either clear from the context or not important.

Definition 1 (*Byzantine Resilience*). Suppose that $0 \le q \le m$, and Δ is a given positive number. Let $v_1, v_2, \ldots, v_m$ be i.i.d. random vectors in $\mathbb{R}^d$, where $v_k \sim G$ with $\mathsf{E}[G] = g$. Let $\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m$ be *attacked* copies of v_k's, such that at least $m-q$ of which are equal to the corresponding v_k's while the rest are arbitrary vectors in $\mathbb{R}^d$. An aggregation rule $\mathrm{Aggr}(\cdot)$ is said to be *Δ-Byzantine resilient* if

$$\mathsf{E}\left[\| \mathrm{Aggr}(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m) - g \|^2\right] \le \Delta. \tag{4}$$

3.3. Clipping mean in FL

In federated learning, mean clipping is a technique used to enhance the robustness of the global model by preventing extreme data from individual clients from disproportionately influencing the overall model updates. The process begins with each client training its local model and computing gradients based on its dataset, which are then sent to the central server. The server performs gradient clipping to limit the magnitude of each client's gradient, ensuring it does not exceed a specified threshold. Once clipped, the server aggregates these gradients and applies mean clipping, where the averaged gradients are constrained within a predefined range to avoid extreme values. Afterward, the server performs weighted aggregation, typically based on the size of each client's dataset, to update the global model. This updated model is then distributed back to all clients for the next round of training. By limiting the impact of any one client's extreme data, mean clipping helps maintain the fairness, security, and robustness of the global model throughout the training process.

4. FL framework with robust aggregation

Nowadays, the aggregation rules based on the FL framework are mainly based on $\mathrm{Krum}(\cdot)$ and $\mathrm{Mean}(\cdot)$. They are usually used in the defense of Byzantine node attacks. However, the aggregation rules based on $\mathrm{Mean}(\cdot)$ often cannot exhibit strong robustness, if it is affected by Byzantine nodes. The attack makes the gradient estimation direction deviate too far from the actual gradient update direction; and it cannot guarantee the robustness of the data. Thus we shall propose trimmed mean-based aggregation rules to resolve this issue.

4.1. Krum

The Krum rule is a strong aggregation rule that attempts to identify an honest computing node, and discards the data of other computing nodes (Blanchard et al., 2017). The data from the identified honest computing node is used in the next round of algorithm. The selection strategy is to find one whose data is closest to that on other computing nodes. In other words, it computes the local sum of squared Euclidean distances to the other candidates, and outputs the one with minimal sum. A notable property of $\mathrm{Krum}(\cdot)$ is shown in Lemma 1.

Lemma 1 (*Blanchard et al., 2017*). *Let $v_1, \ldots, v_m \in \mathbb{R}^d$ be any i.i.d. random vectors such that $v_k \sim G$ with $\mathsf{E}[G] = g$ and $\mathsf{E}\left[\|G-g\|^2\right] \le V$. Let $\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m$ be attacked copies of v_k's, where up to q of them are Byzantine. If $2q+2<m$, then $\mathrm{Krum}(\cdot)$ is Δ_0-Byzantine resilient, where*

$$\Delta_0 = \left(6m - 6q + \frac{4q(m-q-2)+4q^2(m-q-1)}{m-2q-2}\right)V. \tag{5}$$

4.2. Mean-based aggregation rules

An important class of aggregation rules is mean-based aggregation rules. Mathematically, an aggregation rule $\mathrm{Aggr}(\cdot)$ is a *mean-based* one if $\mathrm{Aggr}(v_1, v_2, \ldots, v_m)$ is guaranteed to be a convex combination of $v_1, v_2, \ldots, v_m$. Roughly speaking, a mean-based rule produces an averaged value of the input data. For instance, the arithmetic mean $\mathrm{Mean}(v_1, v_2, \ldots, v_m) = \frac{1}{m}\sum_{k=1}^m v_k$ is the simplest mean-based aggregation rule. However, simple mean-based aggregation rules are in general *not* robust under Byzantine attacks, in the sense that they do *not* satisfy Byzantine resilience conditions (Yin et al., 2018).

In the following, we show that by carefully handling the data, certain mean-based rules can satisfy Byzantine resilience conditions. We first introduce the concept of *b-trimmed means* based on order statistics as in Definition 1. This concept is closely related to the coordinate-wise trimmed mean defined in Yin et al. (2018).

Definition 2. Let $u_1, u_2, \ldots, u_m$ be scalars whose order statistics is

$$u_{(1)} \le u_{(2)} \le \cdots \le u_{(m)}. \tag{6}$$

For an integer b with $0 \le b < m/2$, the b-trimmed mean of $u_1, u_2, \ldots, u_m$ is defined as

$$\mathrm{Tmean}_b(u_1, u_2, \ldots, u_m) = \frac{1}{m-2b}\sum_{k=b+1}^{m-b} u_{(k)}. \tag{7}$$

For d-vectors $v_1, v_2, \ldots, v_m \in \mathbb{R}^d$, b-trimmed mean $\mathrm{Tmean}_b(v_1, v_2, \ldots, v_m) \in \mathbb{R}^d$ is defined in the component-wise sense, i.e.,

$$\mathrm{Tmean}_b(v_1, v_2, \ldots, v_m) = \sum_{i=1}^d e_i \cdot \mathrm{Tmean}_b\big(\langle v_1, e_i\rangle, \langle v_2, e_i\rangle, \ldots, \langle v_m, e_i\rangle\big), \tag{8}$$

where e_i is the ith column of the $d \times d$ identity matrix.

Intuitively, aggregation rules using b-trimmed means have opportunities to discard abnormal values from the input data. Lemma 2 shows that b-trimmed means are usually within a meaningful range under mild assumptions.

Lemma 2. *Let $u_1, u_2, \ldots, u_m$ be scalars whose order statistics is*

$$u_{(1)} \le u_{(2)} \le \cdots \le u_{(m)}, \tag{9}$$

and $\tilde{u}_1, \tilde{u}_2, \ldots, \tilde{u}_m$ be attacked copies of $u_1, u_2, \ldots, u_m$ with up to q Byzantine elements. If the order statistics of $\tilde{u}_k$'s is

$$\tilde{u}_{(1)} \le \tilde{u}_{(2)} \le \cdots \le \tilde{u}_{(m)}, \tag{10}$$

then for $q \le b < m/2$, we have

$$u_{(k-b)} \le u_{(k-q)} \le \tilde{u}_{(k)} \le u_{(k+q)} \le u_{(k+b)}, \qquad (b+1 \le k \le m-b). \tag{11}$$

Proof. Let $\tilde{u}_{(k_0)}$ be the largest non-Byzantine element in $\{\tilde{u}_{(1)}, \tilde{u}_{(2)} \ldots, \tilde{u}_{(k)}\}$. Since the number of Byzantine elements is less than or equal to q, we have $\tilde{u}_{(k)} \ge \tilde{u}_{(k_0)} \ge u_{(k-q)} \ge u_{(k-b)}$ if $k \ge b+1$. The proof of $\tilde{u}_{(k)} \le u_{(k+q)} \le u_{(k+b)}$ for $k \le m-b$ is similar. □

With the help of Lemma 2, we can prove that $\mathrm{Tmean}_b(\cdot)$ is Byzantine resilient when b is greater or equal to the number of Byzantine nodes. The result is formulated as Theorem 1.

Theorem 1. *Let $v_1, \ldots, v_m \in \mathbb{R}^d$ be i.i.d. random vectors such that $v_k \sim G$ with $\mathsf{E}[G] = g$ and $\mathsf{E}[G-g]^2 \le V$. Let $\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m$ be attacked copies of v_k's, where up to q of them are Byzantine. If $q \le b < m/2$, then $\mathrm{Tmean}_b(\cdot)$ is Δ_1-Byzantine resilient, where*

$$\Delta_1 = \frac{mV}{(m-b)^2}. \tag{12}$$

Proof. See Appendix. □

4.3. Tmean time complexity analysis

The exact time complexity for evaluating $\mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m)$ has no closed-form expression in terms of (m, b, d). However, by sorting in each dimension, we can evaluate $\mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m)$ using $O(dm\log m)$ operations. The cost is almost linear in practice, and is not much more expensive than evaluating $\mathrm{Mean}(\cdot)$. Specifically, Tmean (Trimmed Mean) is a robust aggregation technique used in federated learning to mitigate the impact of outliers by discarding the largest and smallest gradient values from clients before calculating the mean. The time complexity of Tmean primarily stems from the sorting step. Specifically, for each position in the gradient vector, Tmean requires sorting the gradients from m clients, which incurs a time complexity of $O(dm\log m)$, where m represents the number of participating clients. After sorting, Tmean discards the top k largest and bottom k smallest values, which can be done in linear time, resulting in a time complexity of $O(dm)$ for this step. Once the outliers are removed, the remaining $m-2k$ values are averaged to compute the trimmed mean, also with a time complexity of $O(dm)$. Thus, the overall time complexity of Tmean is dominated by the sorting step, yielding a complexity of $O(dm\log m)$.

In comparison, the standard mean aggregation method simply computes the weighted average of all client gradients without the need for sorting, resulting in a time complexity of $O(dm)$. Therefore, Tmean introduces an additional computational overhead due to the sorting operation, making its time complexity higher than the standard mean method. However, in practice, the number of clients m in federated learning is often not very large (e.g., hundreds to thousands), meaning that the extra overhead from sorting in Tmean is usually manageable and may be considered negligible. While the theoretical time complexity of Tmean is higher than the basic mean aggregation, this additional overhead is often small enough to have a limited impact on the overall system performance, making Tmean a feasible and practical method for enhancing the robustness of federated learning models against outliers.

5. Convergence analysis

In this section, we conduct a convergence analysis for SGD using a Byzantine resilient aggregation rule, such as $\mathrm{Krum}(\cdot)$ and $\mathrm{Tmean}(\cdot)$. Since a general convex optimization setting is used, we quote some classic convex analysis theories (see, e.g.,), as shown in Lemma 3.

Lemma 3. *Let $F : \mathbb{R}^d \to \mathbb{R}$ be a μ-strongly convex and L-smooth function. Then for $x, y \in \mathbb{R}^d$ and $\alpha \in [0, 1/2]$, one has*

$$\langle \nabla F(x) - \nabla F(y), x - y \rangle \geq \alpha\mu \|x - y\|^2 + \frac{1-\alpha}{L} \|\nabla F(x) - \nabla F(y)\|^2. \tag{13}$$

Proof. The μ-strong convexity and L-smoothness of $F(x)$ implies

$$\langle \nabla F(x) - \nabla F(y), x - y \rangle \geq \mu \|x - y\|^2,$$

$$\langle \nabla F(x) - \nabla F(y), x - y \rangle \geq \frac{1}{L} \|\nabla F(x) - \nabla F(y)\|^2.$$

The conclusion is a simple convex combination of these inequalities. See the Appendix. □

Using Lemma 3 we establish the convergence of SGD as shown in Theorem 2. The expected error bound consists of a linear convergence term similar to that for usual gradient descent methods (Fraiman & Muniz, 2004), and a term caused by randomness. The theorem also suggests theoretically the largest step size: $\gamma = L^{-1}$.

Theorem 2. *Suppose that the SGD method with (3) is adopted to solve problem (1), where $F : \mathbb{R}^d \to \mathbb{R}$ is a μ-strongly convex and L-smooth function. Let $v_1^{(t)}, \ldots, v_m^{(t)}$ be local gradients at tth iteration, and $\tilde{v}_1^{(t)}, \ldots, \tilde{v}_m^{(t)}$ be the corresponding attacked copies. If the aggregation rule $\mathrm{Aggr}(\cdot)$ is Δ-Byzantine resilient, and the step size γ satisfies $\gamma \leq L^{-1}$, then*

$$\mathsf{E}\left[\|x^{(t)} - x^*\|\right] \leq \eta^t \|x^{(0)} - x^*\| + \frac{1 + \sqrt{1 - \gamma\mu}}{\mu} \sqrt{\Delta}, \tag{14}$$

where x^* is the exact solution, and

$$\eta = \sqrt{1 - \gamma\mu} \geq \sqrt{1 - \frac{\mu}{L}}. \tag{15}$$

Proof. See Appendix. □

We remark that if we allow a general $\alpha \in [0, 1]$ in the proof of Theorem 3 and adjust the corresponding step size as $\gamma \leq 2(1-\alpha)L^{-1}$, the decay ratio η is then bounded through

$$\eta = \sqrt{1 - 2\alpha\gamma\mu} \geq \sqrt{1 - 4\alpha(1-\alpha)\frac{\mu}{L}} \geq \sqrt{1 - \frac{\mu}{L}}. \tag{16}$$

The optimal lower bound is achieved when $\alpha = 1/2$ and $\gamma = L^{-1}$.

For general non-strongly convex functions and non-convex functions, we provide the following convergence guarantee.

Theorem 3. *Assume that $F(x)$ is L_F-smooth and potentially non-convex, where $0 < L_F$. We take $\gamma \leq \frac{1}{L_F}$. In any iteration t, the correct gradients are $v_i^t = \nabla F_i\left(x^t\right)$. Using any (classic or dimensional) Δ-Byzantine-resilient aggregation rule with corresponding assumptions, we obtain linear convergence with a constant error after T iterations with synchronous SGD:*

$$\frac{\sum_{i=0}^{T-1} \mathbb{E}\left\|\nabla F\left(x^i\right)\right\|^2}{T} \leq \frac{2}{\gamma T}\left[F\left(x^0\right) - F\left(x^*\right)\right] + \Delta. \tag{17}$$

Proof. See Appendix. □

6. Experiments

In this section we verify the robustness and convergence of Tmean aggregation rules by experiments. We use a multi-layer perceptron with two hidden layers to handwritten digits classification on the MNIST data set, in $m = 20$ worker processes, we repeat each experiment for ten times and take the average value. Table 1 shows the detailed network structures of the MLP used in our experiments. Then, we conduct recognition on convolutional neural network in the Cifar10 dataset, repeat each experiment for ten times and report the averaged value. In our experiments, we use $\mathrm{Mean}(\cdot)$ *without* Byzantine attack as a reference.

Table 1
Structure of MLP.

Layer type	Flatten->fc1->relu1->fc2->relu2->fc3->softmax
Parameters	Null->#output128->null>#output128->null->#output10->null
Previous Layer	data->flatten->fc1->relu1->fc2->relu2->fc3

Table 2
Experiment summary.

Data set	#train	#test	#rounds	γ	Batch size	Evaluation metric
MNIST	60,000	10,000	600	0.1	32	Top-1 accuracy
CIFAR10	50,000	10,000	3000	5×10^{-4}	128	Top-3 accuracy

We use top-1 or top-3 accuracy on testing sets as the evaluation metrics. Table 2 lists the details of the data set and the default hyper-parameters for the corresponding model. We also carried out experiments under the Cifar 100 dataset, and also applied the settings of Cifar 10, but the experimental results were not satisfactory, and we concluded that under the Cifar 100 dataset, the anti-Byzantine method based on Tmean and Krum, and geometric median showed similar resistance to Gaussian attacks and other attacks, which may have a lot to do with the classification terms of the dataset images. We also carried out experiments with this configuration in Cifar 100, and found that the experimental effect was not ideal. After iterative rounds, the accuracy was even less than 0.4, We adjusted the parameters, but it could not significantly affect the results. Taking into account the data set, the mean interception-based Byzantine aggregation method does not have strong robustness under the Cifar 100 data set, but it is worth noting that the mean interception-based aggregation method is still more accurate than other aggregation methods, although not obvious, but also improved, with an average of 0.02–0.04.

Our experiments are implemented by Python 3.7.4 with packages TensorFlow 2.4.1 and NumPy 1.19.2, on a physical node Intel i7-9700 CPU with 32 GB of memory and two NVIDIA GeForce RTX 1080 Ti.

6.1. Experiment setting

In our experiment setting, 6 out of the 20 vectors are Byzantine vectors (i.e., $m = 20$, $q = 6$), different values of b will affect different convergence performance. For the MNIST dataset, we take $b = 6$, $b = 8$ and $b = 10$ to train the experiment; and we take $b = 8$ and $b = 10$ on the Cifar10 dataset to conduct recognition task.

6.2. Gaussian attack

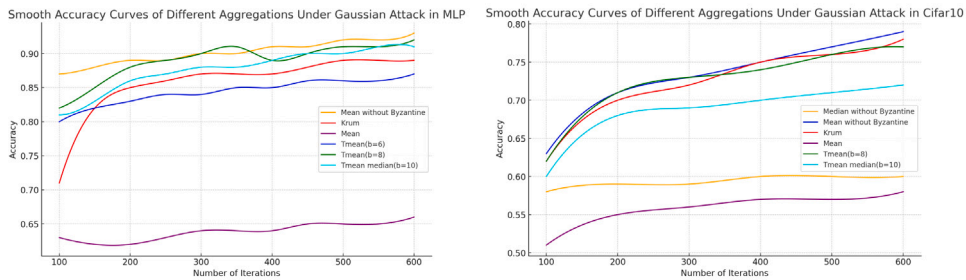
In Gaussian attack experiment, We use the Gaussian random vector with zero mean and isotopic covariance matrix, with standard deviation 200 instead of some correct gradient vectors.

From Table 3 for the MNIST dataset, we find that compared with $\mathrm{Mean}(\cdot)$ without Byzantine, $\mathrm{Tmean}_b(\cdot)$ works well, and is more robust when $b = 8$. However, without trimming the data, $\mathrm{Mean}(\cdot)$ is not robust under Byzantine attack. The method of $\mathrm{Krum}(\cdot)$ is also robust, while it is relatively weaker than $\mathrm{Tmean}(\cdot)$. We can see that the mean trimmed method by selecting the appropriate b is almost the same as when it is not attacked by Gaussian attack, and the accuracy difference between them is not more than 2.5 percent.

For the Cifar10 dataset, we set $b = 8$ and $b = 10$. From Table 4, the aggregation rule based on the $\mathrm{Mean}(\cdot)$ is not as robust as other aggregation rules, but after trimming the $\mathrm{Mean}(\cdot)$, the output is more robust than before. Under the Cifar dataset, we can see that when b

Table 3
Accuracy of different aggregations under Gaussian attack in MLP.

Aggregation rule	Number of iterations										
	100	150	200	250	300	350	400	450	500	550	600
Mean without Byzantine	0.87	0.88	0.89	0.89	0.90	0.90	0.91	0.91	0.92	0.92	0.93
Krum	0.71	0.82	0.85	0.86	0.87	0.87	0.88	0.89	0.88	0.89	0.89
Mean	0.63	0.62	0.62	0.63	0.64	0.64	0.64	0.65	0.65	0.65	0.66
Tmean($b = 6$)	0.80	0.82	0.83	0.84	0.84	0.85	0.85	0.86	0.86	0.86	0.87
Tmean($b = 8$)	0.82	0.85	0.88	0.89	0.89	0.91	0.89	0.90	0.91	0.91	0.92
Tmean $median(b = 10)$	0.81	0.83	0.86	0.87	0.88	0.89	0.89	0.90	0.90	0.91	0.91



(a) Aggregation under Gaussian Attack in MLP (b) Aggregation under Gaussian Attack in Cifar 10

Fig. 3. Fig. 3(a) is Aggregations under Gaussian Attack in MLP. Fig. 3(b) represents is Aggregations under Gaussian Attack in Cifar 10.

Table 4
Accuracy of different aggregations under Gaussian attack in Cifar10.

Aggregation rule	Number of iterations					
	500	1000	1500	2000	2500	3000
$Median$ without Byzantine	0.58	0.59	0.59	0.60	0.60	0.60
Mean without Byzantine	0.63	0.71	0.73	0.75	0.77	0.79
Krum	0.62	0.70	0.72	0.75	0.76	0.78
Mean	0.51	0.55	0.56	0.57	0.57	0.58
Tmean($b = 8$)	0.62	0.71	0.73	0.74	0.76	0.77
Tmean $median(b = 10)$	0.60	0.68	0.69	0.70	0.71	0.72

takes 8, the accuracy achieved by the mean interception algorithm is not much different from Krum(·), they are no more than 3 percentage point, which also proves that Krum(·) aggregation law is extremely adaptable to Gaussian attacks. For Cifar 100 data set, all the existing aggregation rules have a general accuracy below 0.3 under Gaussian attack, and no aggregation method shows better results. Due to the poor effect, the results are not listed in the paper.

6.3. Omniscient attack

Assuming that the attackers (Byzantine nodes) know all the correct gradients information. For each Byzantine gradient vector, all correct gradients of the gradient are replaced by their negative sum. In other words, this attack attempts to make the parameter server go into the opposite direction.

For the MNIST dataset, we see from Table 5 that when the Byzantine node makes the gradient update direction deviate from the maximum actual update direction, Mean(·) cannot be robust. On the contrary, Krum(·) is more robust, while Tmean(·) shows only weak robustness. Because trim is a part of the data intercepted from the mean value for aggregation, it removes the head and tail parts of the entire data set; such that it does not decrease much compared to the overall set mean accuracy when it is attacked by Byzantine. Regardless of the value of b, after receiving an omniscient attack, it can achieve far less precision than Krum(·). However, as the number of iterations increases, the gap between the mean trimmed algorithm and the Krum(·) algorithm is decreasing, but it is still about 9 percent less accurate.

For the Cifar10 dataset, Table 6 shows that Krum(·) is more robust than others. Mean-based aggregation behaved badly in data robustness under omniscient attack. If we trim Mean(·), it can be clearly seen that, the output becomes more robust. However, under this attack, Tmean(·) is not as robust as Krum(·). The Cifar10 dataset and the Mnist dataset are roughly similar, but the difference in accuracy based on Krum and mean interception is amplified. For the Cifar 100 dataset, at omniscient attack, all existing aggregation rules have a general accuracy of about 0.25, and the aggregation method using Krum(·) is slightly higher than the other aggregation methods, with an accuracy of about 2 percent.

6.4. General attack with multiple servers

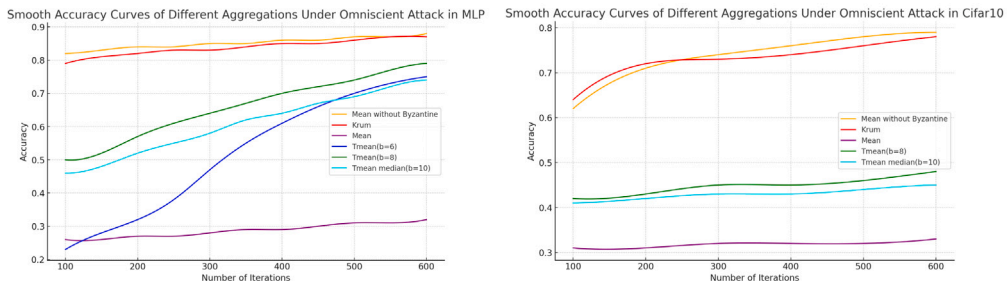
We evaluate the robust aggregation rules under a more general and realistic type of attack. It is very popular to partition the parameters into disjoint subsets, and use multiple server nodes to store and aggregate them. We assume that the parameters are evenly partitioned and assigned to the server nodes. The attacker picks one server, and manipulates any floating number by multiplying -10^{20} with probability of 0.05%. Because the attacker randomly manipulates the values, with the goal that in some iterations the assumptions/prerequisites of the robust aggregation rules are broken, which crashes the training. Such an attack requires less global information, and can be concentrated on one single server, which makes it more realistic and easier to implement.

For the MNIST dataset, in Table 7 we evaluate the performance of all robust aggregation rules under multiple serves attack. The number of servers is 20. For Krum(·) and Mean(·), we set the estimated Byzantine number $q = 6$. We find that Krum(·) and not passed. The intercepted Mean(·) aggregation rules cannot be robust. In this case, they cannot converge to the global optimum. On the other hand, for Tmean(·), different values of b will affect different convergence performance. And if we take $b = 6$, it can also behave robustly.

For the Cifar10 dataset in Table 8, we can find the advantage of the Tmean(·). Neither Krum(·) nor Mean(·) is robust, while Tmean(·) can make the learning robust. It is worth noting that when we use the mean intercept aggregation rule. If we take b to 8, when receiving a multi-server node attack, it differs from the accuracy of not being attacked by only 1.5 percent. And in this attack, the method we propose is much more accurate than the Krum(·) -based algorithm which also shows

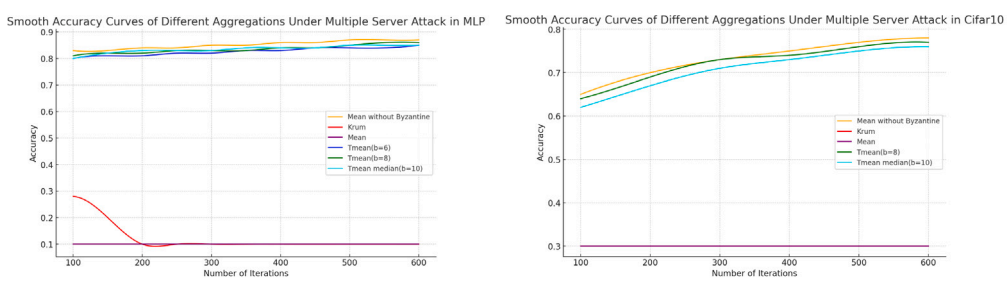
Table 5
Accuracy of different aggregations under omniscient attack in MLP.

Aggregation rule	Number of iterations										
	100	150	200	250	300	350	400	450	500	550	600
Mean without Byzantine	0.82	0.83	0.84	0.84	0.85	0.85	0.86	0.86	0.87	0.87	0.88
Krum	0.79	0.81	0.82	0.83	0.83	0.84	0.85	0.85	0.86	0.87	0.87
Mean	0.26	0.26	0.27	0.27	0.28	0.29	0.29	0.30	0.31	0.31	0.32
Tmean($b = 6$)	0.23	0.28	0.32	0.38	0.47	0.55	0.61	0.66	0.70	0.73	0.75
Tmean($b = 8$)	0.50	0.52	0.57	0.61	0.64	0.67	0.70	0.72	0.74	0.77	0.79
Tmean $median(b = 10)$	0.46	0.48	0.52	0.55	0.58	0.62	0.64	0.67	0.69	0.72	0.74



(a) Aggregation under Omniscient Attack in MLP (b) Aggregation under Omniscient Attack in Cifar 10

Fig. 4. Fig. 4(a) is Aggregations under Omniscient Attack in MLP. Fig. 4(b) represents is Aggregations under Omniscient Attack in Cifar 10.



(a) Aggregation under multiple serves Attack in MLP (b) Aggregation under multiple serves Attack in Cifar 10

Fig. 5. Fig. 5(a) is Aggregations under multiple serve Attack in MLP. Fig. 5(b) represents is Aggregations under multiple serves Attack in Cifar 10.

Table 6
Accuracy of different aggregations under omniscient attack in Cifar10.

Aggregation rule	Number of iterations					
	500	1000	1500	2000	2500	3000
Mean without Byzantine	0.62	0.71	0.74	0.76	0.78	0.79
Krum	0.64	0.72	0.73	0.74	0.76	0.78
Mean	0.31	0.31	0.32	0.32	0.32	0.33
Tmean($b = 8$)	0.42	0.43	0.45	0.45	0.46	0.48
Tmean $median(b = 10)$	0.41	0.42	0.43	0.43	0.44	0.45

that the algorithm we proposed can perfectly adapt to the multi-server attack. It is worth noting that under this kind of attack, the aggregation method based on mean interception is almost 10 percent more accurate than other aggregation methods in Cifar 100 data set, although the accuracy is only 0.4. However, compared with other aggregation rules, the aggregation rule based on this Tmean(·) is obviously better in the face of slightly complex data sets. But for the whole, the performance is still not ideal.

6.5. Experiment conclusion

From the above experiments, we find that it is difficult for Mean(·) to ensure the robustness of the data when it is attacked by any Byzantine node. When multiple server nodes are used at the same time, Krum(·) also becomes vulnerable. However, after the data is properly trimmed, Tmean(·) can maintain data robustness. Under omniscient attacks, Tmean(·) is not as robust as Krum(·). But Tmean(·) still improves a lot compared to Mean(·) in this case. The data robustness of Tmean(·) is also related to the value of b. We find that when the value of b is closer to $m/2$, it can achieve stronger robustness.

7. Conclusions

We analyzed the aggregation rules of Tmean(·). We demonstrated that the effectiveness of our approaches can make the FL model robust to Byzantine attacks. We used three different Byzantine node attacks to show that the original data set can be more robustly handled after partial data trimming and averaging operations. In the experimental process, we used the Mnist and Cifar10 datasets, and the algorithms proposed under both datasets were greatly improved compared to the traditional median and mean methods. Especially in multi-server attacks, which solved the problem of not being effectively defended by

Table 7
Accuracy of different aggregations under multiple server attack in MLP.

Aggregation rule	Number of iterations										
	100	150	200	250	300	350	400	450	500	550	600
Mean without Byzantine	0.83	0.83	0.84	0.84	0.85	0.85	0.86	0.86	0.87	0.87	0.87
Krum	0.28	0.20	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Mean	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Tmean($b = 6$)	0.80	0.81	0.81	0.82	0.82	0.83	0.83	0.84	0.84	0.84	0.85
Tmean($b = 8$)	0.81	0.82	0.82	0.83	0.83	0.83	0.84	0.84	0.85	0.86	0.86
Tmean $median(b = 10)$	0.80	0.82	0.83	0.83	0.83	0.84	0.84	0.84	0.85	0.85	0.85

Table 8
Accuracy of different aggregations under multiple server attack in Cifar10.

Aggregation rule	Number of iterations					
	500	1000	1500	2000	2500	3000
Mean without Byzantine	0.65	0.70	0.73	0.75	0.77	0.78
Krum	0.30	0.30	0.30	0.30	0.30	0.30
Mean	0.30	0.30	0.30	0.30	0.30	0.30
Tmean($b = 8$)	0.64	0.69	0.73	0.74	0.76	0.77
Tmean $median(b = 10)$	0.62	0.67	0.71	0.73	0.75	0.76

statistical aggregation rules. In the Appendix section of the article, we have performed a convergence analysis of the algorithm. And proved that the proposed method can converge in both strong and non-strong convex environments by means that the difference is constantly less than the convergence term, and the results are optimizable.

This work focuses on Tmean(·). As a new statistical aggregation algorithm, the mean trimmed method has strong robustness under Gaussian attack, omniscient attack and multi-server attack. However, the mean trimmed method may be relatively simple in calculation. So it still has strong space for expansion under Byzantine node attackS. On the other hand, we can try to use geometric median and coding correlation strategy to carry out more in-depth research under the aggregation rules based on statistical methods. In future work we plan to refine the trimming range of Tmean(·) and prove its convergence in a non-convex environment. At the same time, we will add momentum on the basis of Tmean(·), or add some constraints to strengthen its robustness.

In addition,in the CIFAR 100 dataset. After 3000 iterations, the accuracy of Tmean(·) aggregation algorithm is between 0.25 and 0.4. It is not advisable to use the simple Tmean(·) aggregation rule and Krum(·) for Byzantine fault tolerance, which is worth further studying. Much of our research works are based on statistical aggregation methods. We will effectively defend against model node attacks by using differential privacy strategies before aggregation, such as by adding noise. This is the difficult point of the current research, with good development prospects.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Zhonglong Zheng reports financial support, article publishing charges, and statistical analysis were provided by National Natural Science Foundation of China. Zhonglong Zheng reports a relationship with National Natural Science Foundation of China that includes: funding grants.

Acknowledgments

This work was supported by National Natural Science Foundation of China No. 62272419.

Appendix

Proof of Theorem 1. We first assume that v_k's, $\tilde{v}_k$'s, and g are all scalars (i.e., $d = 1$), with variance $V = \sigma^2$. Let $v_{(1)} \le v_{(2)} \le \cdots \le v_{(m)}$ and $\tilde{v}_{(1)} \le \tilde{v}_{(2)} \le \cdots \le \tilde{v}_{(m)}$, respectively, be order statistics of $v_1, v_2, \ldots, v_m$ and $\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m$. Notice that

$$\begin{aligned}
\mathsf{E}\left[\left(\frac{1}{m-2b}\sum_{k=1}^{m-2b} v_{(k)} - g\right)^2\right] &= \frac{1}{(m-2b)^2}\mathsf{E}\left[\sum_{k=1}^{m-2b}(v_{(k)} - g)^2\right] \\
&\le \frac{1}{(m-2b)^2}\mathsf{E}\left[\sum_{k=1}^{m}(v_{(k)} - g)^2\right] \\
&= \frac{1}{(m-2b)^2}\mathsf{E}\left[\sum_{k=1}^{m}(v_i - g)^2\right] \\
&= \frac{m\sigma^2}{(m-2b)^2}.
\end{aligned}$$

Similarly, we have

$$\mathsf{E}\left[\left(\frac{1}{m-2b}\sum_{k=2b+1}^{m} v_{(k)} - g\right)^2\right] = \frac{m\sigma^2}{(m-2b)^2}.$$

It follows from Lemma 2 that

$$\frac{1}{m-2b}\sum_{k=1}^{m-2b} v_{(k)} - g \le \mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m) - g \le \frac{1}{m-2b}\sum_{k=2b+1}^{m} v_{(k)} - g.$$

Therefore

$$\begin{aligned}
&\mathsf{E}\left[(\mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m) - g)^2\right] \\
&\le \max\left\{\mathsf{E}\left[\left(\frac{1}{m-2b}\sum_{k=1}^{m-2b} v_{(k)} - g\right)^2\right], \mathsf{E}\left[\left(\frac{1}{m-2b}\sum_{k=2b+1}^{m} v_{(k)} - g\right)^2\right]\right\} \\
&\le \frac{m\sigma^2}{(m-2b)^2}.
\end{aligned}$$

Now we consider the general case—v_k's, $\tilde{v}_k$'s, and g are d-vectors, with variance $V = \sum_{i=1}^{d}\sigma_i^2$, where

$$\mathsf{E}\left[\langle G - g, e_i\rangle^2\right] \le \sigma_i^2.$$

Using the result for the scalar case, we have

$$\mathsf{E}\left[\langle \mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m) - g, e_i\rangle^2\right] \le \sum_{i=1}^{d}\frac{m\sigma_i^2}{(m-2b)^2}.$$

Thus we conclude that

$$\mathsf{E}\left[\|\mathrm{Tmean}_b(\tilde{v}_1, \tilde{v}_2, \ldots, \tilde{v}_m) - g\|^2\right] \le \sum_{i=1}^{d}\frac{m\sigma_i^2}{(m-2b)^2} = \frac{mV}{(m-2b)^2}. \quad \square$$

Proof of Lemma 3.

$$2\langle \nabla F(x) - \nabla(y), x - y\rangle \geqslant \mu\|x - y\|^2 + \frac{1}{L}\|\nabla F(x) - \nabla F(y)\|^2$$

$$\langle \nabla F(x) - \nabla F(y), x - y\rangle \geqslant \frac{\mu}{2}\|x - y\|^2 + \frac{1}{2L}\|\nabla F(x) - \nabla F(y)\|^2$$

$$\alpha \in \left[0, \frac{1}{2}\right], \ \frac{\mu}{2} \geqslant \frac{\alpha}{2}, \ \frac{1}{2L} \geqslant \frac{1-\alpha}{L}$$

$$\begin{aligned}
&\frac{\mu}{2}\|x - y\|^2 + \frac{1}{2L}\|\nabla F(x) - \nabla F(y)\|^2 \\
&\geqslant \alpha\mu\|x - y\|^2 + \frac{1-\alpha}{L}\|\nabla F(x) - \nabla F(y)\|^2 \quad \square
\end{aligned}$$

Data availability

Data will be made available on request.

References

Alistarh, D., Allen-Zhu, Z., & Li, J. (2018). Byzantine stochastic gradient descent. arXiv preprint arXiv:1803.08917.

Blanchard, P., Mhamdi, E. M. E., Guerraoui, R., & Stainer, J. (2017). Byzantine-tolerant machine learning. In *Conference and workshop on neural information processing systems* (pp. 118–128).

Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT* (pp. 177–186). Springer.

Castro, M., Liskov, B., et al. (1999). Practical Byzantine fault tolerance. *vol. 99*, In *OSDI* (pp. 173–186).

El-Mhamdi, E.-M., Guerraoui, R., Guirguis, A., Hoang, L. N., & Rouault, S. (2020). Genuinely distributed Byzantine machine learning. In *Proceedings of the 39th symposium on principles of distributed computing* (pp. 355–364).

El Ouadrhiri, A., & Abdelhadi, A. (2022). Differential privacy for deep and federated learning: A survey. *IEEE Access*, *10*, 22359–22380.

Fraiman, R., & Muniz, G. (2004). Trimmed means for functional data. *Test*.

Helmers, R., Jing, B.-Y., Qin, G., & Zhou, W. (2024). *Saddlepoint approximations to the trimmed mean* (pp. 465–501). Bernoulli.

Hu, T., & Sung, S. Y. (2004). A trimmed mean approach to finding spatial outliers. In *International conference on machine learning* (pp. 79–95). Intelligent Data Analysis.

Jin, R., Huang, Y., He, X., Wu, T., & Dai, H. (2020). Stochastic-sign SGD for federated learning with theoretical guarantees. arXiv preprint arXiv:2002.10940.

Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., & Suresh, A. T. (2020). Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning* (pp. 5132–5143). PMLR.

Konečnỳ, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., & Bacon, D. (2016). Federated learning: Strategies for improving communication efficiency. arXiv preprint arXiv:1610.05492.

Lamport, L., Shostak, R., & Pease, M. (2019). The Byzantine generals problem. In *Concurrency: the works of leslie lamport* (pp. 203–226).

Li, M., Andersen, D. G., Smola, A. J., & Yu, K. (2014). Communication efficient distributed machine learning with the parameter server. *Advances in Neural Information Processing Systems*, *27*, 19–27.

Lugosi, G., & Mendelson, S. (2021). Robust multivariate mean estimation: the optimality of trimmed mean. arXiv.

Ranjan, P., Gupta, A., Coro, F., & Das, S. K. (2023). Robust federated learning against backdoor attackers. In *IEEE INFOCOM 2023-IEEE conference on computer communications workshops (INFOCOM WKSHPS)* (pp. 1–6). IEEE.

Wei, W., Liu, L., Loper, M., Chow, K.-H., Gursoy, M. E., Truex, S., et al. (2020). A framework for evaluating gradient leakage attacks in federated learning. arXiv preprint arXiv:2004.10397.

Yin, D., Chen, Y., Kannan, R., & Bartlett, P. (2018). Byzantine-robust distributed learning: Towards optimal statistical rates. In *International conference on machine learning* (pp. 5650–5659). PMLR.

Yin, D., Chen, Y., Kannan, R., & Bartlett, P. (2019). Defending against saddle point attack in Byzantine-robust distributed learning. In *International conference on machine learning* (pp. 7074–7084). PMLR.

Zhao, B., Sun, P., Fang, L., Wang, T., & Jiang, K. (2021). FedCom: A Byzantine-robust local model aggregation rule using data commitment for federated learning. In *USENIX Security Symposium*.

Zhu, H., Wang, R., & Liu, X. (2021). Blockchain for federated learning: Security and privacy perspectives. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(11), 4825–4836.